

Smart Home für betreutes Wohnen

Ein verteilter, ereignisgesteuerter Demonstrator auf MQTT-Basis

Joel Deffner

Heterogeneous Computing, SS 2026

Was gebaut wurde und warum

Szenario: Wohnung im betreuten Wohnen. Sensorik soll Pflegekräfte bei kritischen Situationen alarmieren (Sturz/SOS, Herd vergessen, nächtliches Verlassen des Bettes).

Aufgabe: Aus Recherche zu Smart-Home-Architekturen zentrale Anforderungen ableiten und einen prototypischen Demonstrator aus *mehreren kommunizierenden Komponenten* umsetzen. Physische Hardware wird softwareseitig simuliert.

Stack

- Docker + Eclipse Mosquitto (MQTT-Broker)
- TypeScript / Node, je Komponente ein Prozess
- Web-Konsole (Express + SSE)

Kernidee

- Kein Gerät spricht ein anderes direkt an
- Alles läuft als Pub/Sub über den Broker
- Logik rein lokal (Edge), keine Cloud

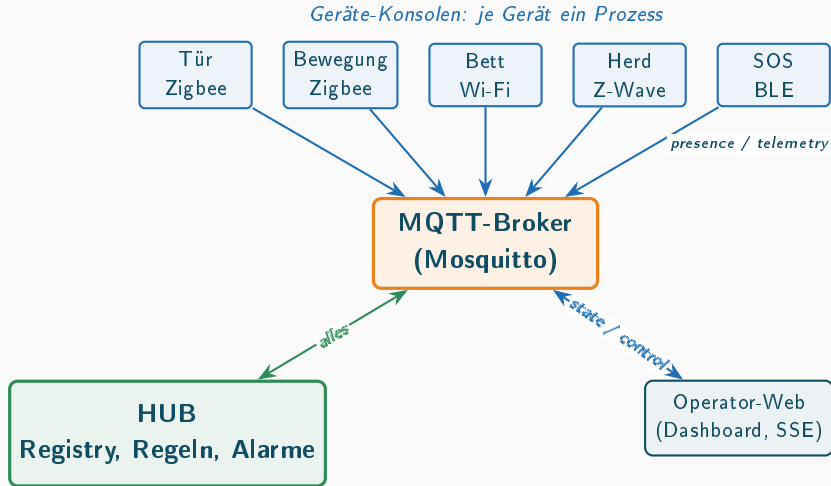
Beobachtungen aus bestehenden Plattformen (Home Assistant, openHAB, ioBroker): zentral / dezentral / hybrid; **MQTT** und CoAP auf Kommunikationsebene; Zigbee, Z-Wave, Thread, Wi-Fi, BLE auf Funkebene; **Matter** als IP-basierte Interoperabilitätsschicht; Trend zur Trennung von **Cloud** und **Edge**.

Abgeleitete Kernanforderungen

- Verteilung
- Skalierbarkeit
- Fehlertoleranz
- Lose Kopplung
- Heterogenität der Geräte
- Erweiterbarkeit

Diese sechs Anforderungen sind der rote Faden für jede Entwurfsentscheidung im Folgenden.

Architektur: Gerät → Broker → Hub



Vier Prozessarten, gekoppelt *nur* über Topics. Fällt eine Komponente aus, laufen die anderen weiter;

Kommunikationsmodell: Publish/Subscribe über Topics

Hierarchische Topics, Wildcards (+, #) ermöglichen Geräteerkennung *ohne* zentrale Konfiguration. QoS 1 (mindestens einmal).

Topic	Inhalt	Eigenschaft
registry/<id>	Geräte-Datensatz + Status	retained
devices/<id>/presence	online / offline	retained, Last-Will
devices/<id>/telemetry	Messwert + Akku + Signal	flüchtig
devices/<id>/command	Steuerbefehl an Gerät	flüchtig
pairing/<serial>	Pairing-Anzeige (neues Gerät)	retained, Last-Will
alarms/<id>	aktiver Alarm	retained
rooms/<id> & hub/status	Räume, Tag/Nacht-Fenster	retained

Ein einziger Vertrag: JSON über MQTT, typisiert in einer geteilten Datei (src/shared/types.ts). Hub und Sensor können nicht auseinanderlaufen.

Nicht-triviale Konzepte

Konzept 1: Fehlertoleranz aus dem Protokoll

Fehlererkennung ist *nicht* aufwendige Anwendungslogik, sondern ergibt sich aus zwei MQTT-Mechanismen.

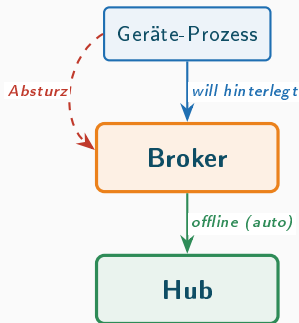
Last-Will (LWT)

Beim Verbinden hinterlegt jedes Gerät beim Broker eine Nachricht `presence = offline` (retained). Stirbt der Prozess (Absturz, Fenster zu, Netz weg), *publiziert der Broker sie automatisch*.

Retained State

Registry, Räume und Status sind retained.

Eine später startende Komponente sieht sofort den kompletten Bestand, ohne jemand abfragen zu müssen



⇒ `device_offline`-Alarm
löst sich beim Reconnect selbst auf

Konzept 2: Onboarding wie bei echter Hardware

Ein neues Gerät bootet **unprovisioniert**, genau wie ab Werk. Es hat noch keine ID und keinen Raum, sondern nur eine Werks-Identität (Serien-Nr. + Setup-PIN).



- Commissioning entweder direkt in der Konsole *oder* per Klick im Web („Geräte, die auf Pairing warten“); Raumwahl per „QR-Scan“ des Raumcodes.
- Abbruch vor dem Commissioning räumt die Pairing-Anzeige automatisch wieder ab (Last-Will).
- Bewusst angelehnt an Zigbee/Z-Wave-Inclusion bzw. Matter-Setup-Codes.

Konzept 3: Regel- und Alarm-Engine

Der Hub trennt sauber: **Regeln** erkennen Situationen, die **Alarm-Engine** verwaltet deren Lebenszyklus.

Regeln

Pro Gerät kleiner Zustand (wann Herd an, wann Bett verlassen, letzte Bewegung im Raum). Eine Situation wird *einmal* beim Eintreten erhoben und *einmal* beim Ende gelöscht, kein Dauerfeuer.

Latching SOS-Alarm

Der Notfallknopf modelliert echtes Verhalten: Loslassen löscht den Alarm *nicht*. Er bleibt kritisch, bis eine Pflegekraft aktiv quittiert.

Regel	Schwere
sos_pressed	kritisch
stove_on_no_motion	kritisch
bed_left_at_night	kritisch
door_open_no_motion	Warnung
device_offline	Warnung (auto)

Alarme sind retained und *dedupliziert*: dasselbe Problem erzeugt keinen zweiten Alarm. Schwellen sind per Env-Var konfigurierbar (kurz für Demos).

Heterogenität hinter einem Datenmodell

Fünf Gerätetypen über vier Funkstandards, jeweils mit realistischem Datenblatt (Hersteller, Modell, Akku %, RSSI, Firmware) und doch ein gemeinsames Nachrichtenformat.

Typ	Produkt (sim.)	Transport	Energie
Tür / Kontakt	Aqara DW-S100	Zigbee	Batterie
Bewegung (PIR)	Philips Hue SML-002	Zigbee	Batterie
Bettbelegung	Emfit QS-Care	Wi-Fi	Netz
Herdwächter	Inirv Guard-Z	Z-Wave	Netz
SOS-Anhänger	CareTech SOS-Pendant	BLE	Batterie

Anforderungen & Demo

Wie die Kernanforderungen konkret gelöst sind

Anforderung	Umsetzung im Prototyp
Verteilung	Eigenständige Prozesse (Hub, Sensoren, Web), gekoppelt nur über den Broker; Logik lokal/verteilt.
Skalierbarkeit	Geräte registrieren sich selbst; das 1. wie das 100. Gerät ist derselbe Vorgang. Topic pro Gerät, Broker verteilt von selbst.
Fehlertoleranz	Last-Will + retained Presence machen aus einem Absturz einen Alarm; Reconnect löst ihn auf. Hub stellt Zustand aus Dateien wieder her.
Robustheit	Auto-Reconnect (2 s); fehlerhafte Nachrichten werden geloggt, nicht weitergereicht; Komponenten laufen unabhängig weiter.
Lose Kopplung	Einzige Schnittstelle: JSON über MQTT. Jede Komponente jederzeit neu startbar.
Heterogenität	Verschiedene Typen, Funkstandards und Energiequellen hinter einem Nachrichtenformat.
Erweiterbarkeit	Neuer Typ = Katalogeintrag + kleine Sensorklasse (+ ggf. Regel). Der SOS-Knopf wurde <i>genau so</i> ergänzt, ohne Änderung an Registry, Konsole oder UI.

Live-Demo: der geplante Ablauf

1. **Pairing** → **Commissioning**: neues Gerät bootet unprovisioniert, erscheint im Web unter „wartet auf Pairing“, Zuweisung zu einem Raum. Es wird live.
2. **Messwert wird Alarm**: Herd anschalten (Taste space); nach kurzer Schwelle ohne Bewegung im Raum erscheint ein roter stove_on-Alarm im Dashboard.
3. **Ausfall**: Geräte-Fenster schließen → Broker sendet Last-Will → Gerät binnen Sekunden offline, device_offline-Warnung. Wieder öffnen → Warnung löst sich automatisch.
4. **SOS**: Notfallknopf drücken → latchender kritischer Alarm, bleibt aktiv bis „Resolve“.
5. **Simulate Night**: Tag/Nacht-Schalter erzwingt das Nachtfenster, damit die Regel „Bett nachts verlassen“ auch tagsüber demonstrierbar ist.

Verifiziert über `pnpm run typecheck` und einen automatisierten Smoke-Test (eingebetteter Broker, kein Docker): Onboarding → Alarm → Offline → Auto-Resolve.

Grenzen (Scope)

- Keine Cloud-Schicht (Fernzugriff, Flottenverwaltung) bewusst ausgeklammert.
- Sicherheit nur skizziert: z.B. kein TLS.
- Ein einzelner Broker und Hub.
- Geräte simuliert, Funkstandards als Metadaten.

Natürliche nächste Schritte

- Authentifizierung am Commissioning-Pfad (der richtige Ort dafür).
- TLS + ACLs pro Gerät auf dem Broker.
- Broker-Cluster / mehrere Hubs für Hochverfügbarkeit.

Vielen Dank für Ihre Aufmerksamkeit.